技术报告:Linux 基础、Git 与 C++ 闯关做法

Linux 基础

第 1 关:找到入口

这关要你交一个文件 `/workspace/flag`,里面的内容是拿密钥把 `/workspace/input` 处理一遍得到的结果,而且要一个字节都不差地比对。

工作区里主要有三样东西:柜子里有个 `key.txt`,写着密钥 `MOON`,注意这个文件其实是 5 个字节,最后多了个换行,真正的密钥是 4 个字母 `MOON`;还有一个 `input.bin`,是 16 个字节的材料;还有一个 `exam`,是个 Python 程序,干的事就是把输入的每个字节和密钥循环异或。

这里有两个坑。第一,程序读的文件叫 `/workspace/input`,但材料叫 `input.bin`,名字对不上,得先复制一下。第二,它的参数是 `-e` 不是 `-k`,而且程序报错只会说一句参数无效,把真正原因藏起来了。

做法一,用程序本身:

```
cp /workspace/input.bin /workspace/input
/workspace/exam -e MOON
cp /workspace/output /workspace/flag
```

做法二,自己把异或算一遍,结果一样:

```
python3 -c
"d=open('/workspace/input','rb').read();k=b'MOON';open('/workspace/flag','wb').w
rite(bytes(v^k[i%len(k)] for i,v in enumerate(d)))"
```

要注意的点:异或有个性质,同一个数异或两次就变回原样,所以看懂程序里那一行 `transform`,就能自己算结果,不用死磕程序怎么调。

第 2 关:两段信号

这关要交的 `flag`,等于 A 段处理结果加上 B 段处理结果,顺序必须是 A 在前、B 在后。A 和 B 靠文件名里的 `signal-A`、`signal-B` 来找,密钥在清单文件里,是唯一一条 `ACCESS_KEY=`

记录。

思路就是用 `find` 和 `grep` 找到两个文件和一个密钥,再复用上一关的程序分别处理,最后拼起来。清单和文件的实际名字按你容器里的情况微调:

```
#!/bin/bash
set -e
cd /workspace

key=$(grep -rhoP 'ACCESS_KEY=\K[^[:space:]]+' . | head -1)
[ -n "$key" ] || { echo "未找到 ACCESS_KEY" >&2; exit 1; }

a=$(find . -type f -name '*signal-A*' | head -1)
b=$(find . -type f -name '*signal-B*' | head -1)
[ -n "$a" ] && [ -n "$b" ] || { echo "信号文件缺失" >&2; exit 1; }

run() {
  cp "$1" /workspace/input
  /workspace/exam -e "$key"
  cp /workspace/output "$2"
}
run "$a" /tmp/partA
run "$b" /tmp/partB

cat /tmp/partA /tmp/partB > /workspace/flag
```

要注意的点:顺序是 A 前 B 后,别乱排。程序每次会先删掉旧的 output,所以两次处理之间只要把 input 覆盖一下就行。

第 3 关:读日志并修复输入

这关要先看 `/var/log/nuscc/exam.log` 里的报错,再修输入。规则是:只能在最后一行文字和结尾换行之间加空格,而且要加得最少。

其实程序早就告诉我们要求了:输入字节数必须能被密钥长度整除。设原来长度是 N,密钥长度是 K,那要补的空格数就是 `(-N) % K`,一个式子就算出来了,不用一个个试。

```
import pathlib

key  = pathlib.Path('/workspace/key.txt').read_bytes().rstrip(b'\n')
data = pathlib.Path('/workspace/input').read_bytes()
```

```
pad   = (-len(data)) % len(key)
fixed = data[:-1] + b' ' * pad + data[-1:]
pathlib.Path('/workspace/input').write_bytes(fixed)
```

之后运行:

```
/workspace/exam -e "$key"
cp /workspace/output /workspace/flag
```

要注意的点:空格一定要插在换行符前面,所以用 `data[:-1]` 加空格再加 `data[-1:]`,这样结尾的换行还在。

第 4 关:route.sh

这关要写个脚本,收两个参数 `MODE` 和 `ITEM`,去调用一个工具 `task-probe`,然后按它的退出码转成结果。

退出码的对应关系是这样:工具退 0,结果写 `receipt:值`,脚本退 0;工具退 11,结果写 `retry:ITEM`,脚本退 75;工具退 23,结果写 `rejected:ITEM`,脚本退 65。参数不对的话,输出 `usage`,退 64,而且不能生成结果文件。

```
#!/bin/bash
set -u

MODE="$1"
ITEM="$2"

usage() { echo "usage"; exit 64; }

[ $# -eq 2 ] || usage
case "$MODE" in ok|retry|fatal) ;; *) usage ;; esac
case "$ITEM" in ''|*[!A-Za-z0-9]*) usage ;; esac
[ ${#ITEM} -le 32 ] || usage

[ -f /workspace/job.env ] && . /workspace/job.env
: "${TASK_TOOL_DIR:?}" "${ROUTE_RESULT:?}" "${ACCESS_TOKEN:?}"

TOOL="$TASK_TOOL_DIR/task-probe"
value=$("$TOOL" --token "$ACCESS_TOKEN" "$MODE" "$ITEM" 2>/dev/null)
rc=$?

case "$rc" in
```

```
  0)  printf 'receipt:%s\n' "$value"  > "$ROUTE_RESULT"; exit 0 ;;
  11) printf 'retry:%s\n'   "$ITEM"   > "$ROUTE_RESULT"; exit 75 ;;
  23) printf 'rejected:%s\n' "$ITEM"  > "$ROUTE_RESULT"; exit 65 ;;
  *)  exit "$rc" ;;
esac
```

要注意的点:结果文件必须正好一行,末尾带换行,而且覆盖旧内容,所以都用 `printf` 加 `>` 重定向。非法参数要在写文件之前就拦下来。工具具体怎么传参数,以它自己的 `help` 为准,可能要微调。

第 5 关:manage.sh

这关要管一个后台进程 `slow-worker`。工作目录由 `WORK_DIR` 指定,PID 记在 `worker.pid` 里。

几个子命令的要求:start 要在 2 秒内返回 0,进程要一直跑,还得出 `heartbeat.log`,而且不能起第二个;status 在跑就输出 `running` 退 0,没跑就输出 `stopped` 退 1;stop 要正常终止并留下 `stopped.log`,没进程也得成功;report 在跑的时候生成 `resource.txt`,没跑就退 1。

```
#!/bin/bash
set -u

WORK_DIR="${WORK_DIR:-.}"
PIDFILE="$WORK_DIR/worker.pid"
WORKER="/opt/recruitment/bin/slow-worker"

usage() { echo "usage"; exit 64; }
[ $# -eq 1 ] || usage
CMD="$1"

is_running() {
  [ -f "$PIDFILE" ] || return 1
  local pid; pid=$(cat "$PIDFILE" 2>/dev/null)
  [ -n "$pid" ] && kill -0 "$pid" 2>/dev/null
}

case "$CMD" in
  start)
    is_running && exit 0
    ( cd "$WORK_DIR" && nohup "$WORKER" >/dev/null 2>&1 & echo $! > "$PIDFILE" )
    exit 0
    ;;
```

```
  status)
    if is_running; then echo running; exit 0; else echo stopped; exit 1; fi
    ;;
  stop)
    if is_running; then
      pid=$(cat "$PIDFILE")
      kill -TERM "$pid" 2>/dev/null
      for _ in $(seq 1 50); do kill -0 "$pid" 2>/dev/null || break; sleep 0.1;
done
      : > "$WORK_DIR/stopped.log"
    fi
    rm -f "$PIDFILE"
    exit 0
    ;;
  report)
    is_running || exit 1
    pid=$(cat "$PIDFILE")
    {
      echo "PID=$pid"
      ps -p "$pid" -o pid,ppid,stat,etime,cmd
      top -b -n 1 -p "$pid"
    } > "$WORK_DIR/resource.txt"
    exit 0
    ;;
  *)
    usage ;;
esac
```

要注意的点:判断进程在不在,用 `kill -0`,它只探测不发信号,最干净。退出用 `TERM` 不用 `KILL`,让进程有机会正常收尾。

第 6 关:setup-account.sh

这关要建一个归档账号。组叫 `archive`,账号叫 `archiver`,UID 在 200 到 999 之间,主组是 archive,不建 home,shell 是 `nologin`。任务文件的属主是 archiver、属组是 archive,权限是 owner 可读写、同组可读、其他人没权限,也就是 `0640`。archiver 能读,visitor 不能读。脚本要能重复跑。

```
#!/bin/bash
set -e

getent group archive >/dev/null || groupadd archive
```

```
if ! getent passwd archiver >/dev/null; then
  uid=200
  while getent passwd "$uid" >/dev/null; do uid=$((uid+1)); done
  useradd -u "$uid" -g archive -M -s /usr/sbin/nologin archiver
else
  usermod -g archive -s /usr/sbin/nologin archiver 2>/dev/null || true
fi

TASK_FILE="${TASK_FILE:-/workspace/task.txt}"
chown archiver:archive "$TASK_FILE"
chmod 640 "$TASK_FILE"
```

要注意的点:每次操作前先用 `getent` 判断在不在,在就复用,不在才建,这样跑多少遍结果都一样。UID 从 200 开始找第一个空着的,别写死。flag 的具体内容要看 `task.txt` 里的说明再补上。

第 7 关:装工具 + 修 C 程序

这关要写个 `setup-build.sh`,把 `build-essential` 装好,还要修 `hello.c`。程序收一个参数,输出 `hello:参数`,再往 `run.log` 里追加 `processed:参数`,两处都要带换行。flag 就是拿 `foundation` 这个参数跑出来的标准输出。

```
#!/bin/bash
set -e
export DEBIAN_FRONTEND=noninteractive

apt-get update
apt-get install -y build-essential
command -v gcc && gcc --version
```

hello.c 内容:

```
#include <stdio.h>

int main(int argc, char **argv) {
    if (argc != 2) return 1;

    printf("hello:%s\n", argv[1]);

    FILE *log = fopen("/workspace/run.log", "a");
    if (log) {
        fprintf(log, "processed:%s\n", argv[1]);
```

```
        fclose(log);
    }
    return 0;
}
```

生成 flag:

```
/workspace/build.sh
./hello foundation > /workspace/flag
```

要注意的点:装软件重复跑没事,所以评测跑两遍也能过。日志一定要用追加模式 `"a"`,不能用写模式 `"w"`,不然会覆盖。

Git 工具链

这几关统一有个目标内容,就是 `notes.txt` 要写成两行,最后一行带换行:

```
local git practice

first local note
```

也就是 `local git practice` 换行、换行、`first local note` 换行。下面都用 `printf` 写,免得换行出错。

1. Git 就在本地

建一个仓库,HEAD 在 `main` 分支,别的先不动。

```
cd /workspace/repo
git init -b main
```

也可以从别处克隆:

```
rm -rf /workspace/repo
git clone /opt/recruitment/git-seed /workspace/repo
cd /workspace/repo
```

2. 看清工作区和暂存区

只改工作区里的 `notes.txt`,不 add、不 commit。

```
cd /workspace/repo
printf 'local git practice\n\nfirst local note\n' > notes.txt
```

3. 留下第一个版本

改完再提交,提交消息必须是 `learn: first local commit`。

```
cd /workspace/repo
printf 'local git practice\n\nfirst local note\n' > notes.txt
git add notes.txt
git commit -m "learn: first local commit"
```

4. 找回历史中的内容

把内容恢复成最初的 version one。两种办法,任选一种。

用 restore:

```
cd /workspace/repo
git restore --source=c0 -- notes.txt
```

用 revert:

```
cd /workspace/repo
git stash -u
git revert --no-edit c1
```

要注意的点:restore 只改工作区,HEAD 和索引不动;revert 需要工作区干净,所以先 stash 一下最稳妥。

5. 让两条历史汇合

main 和 topic 各改各的文件,各提交一次,再把 topic 合进 main。

```
cd /workspace/repo

git checkout main
```

```
printf 'main ready\n' > main.txt
git add main.txt && git commit -m "main ready"

git checkout topic
printf 'topic ready\n' > topic.txt
git add topic.txt && git commit -m "topic ready"

git checkout main
git merge topic
```

要注意的点:两条分支改的是不同文件,合并的时候不会冲突,出来的就是干净的双亲合并。

6. 完成可逆修改

先在 trial 分支改 notes.txt 并提交,合进 main,再用 revert 反向撤销。

```
cd /workspace/repo

git checkout -b trial
printf 'local git practice\n\nfirst local note\n' > notes.txt
git add notes.txt
git commit -m "learn: first local commit"

git checkout main
git merge trial
git revert --no-edit HEAD
```

要注意的点:用 revert 而不是 reset,历史链才完整,就是 c0、修改、反向提交这样一路留痕。

C/C++ 必修

这四关的真实源码截图里没有,所以下面给的是标准通用的写法,考点都说清楚,实际提交时按原题源码补全就行。

1. 读数统计

考的是防溢出和边界。用 `long long` 存和,min 和 max 要初始化好。

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    long long n = 0, sum = 0;
    long long mn = LLONG_MAX, mx = LLONG_MIN;
    long long x;
    while (cin >> x) {
        ++n; sum += x;
        mn = min(mn, x); mx = max(mx, x);
    }
    if (n == 0) { cout << "0 0\n"; return 0; }
    cout << n << " " << sum << " " << mn << " " << mx << "\n";
    return 0;
}
```

2. 设备巡检记录

考的是用 struct 封装数据,再把过滤、统计、查询拆成函数。

```
#include <bits/stdc++.h>
using namespace std;

struct Device { string id; int status; double value; };

vector<Device> filter_by_status(const vector<Device>& ds, int st) {
    vector<Device> r;
    for (auto& d : ds) if (d.status == st) r.push_back(d);
    return r;
}
double sum_value(const vector<Device>& ds) {
    double s = 0; for (auto& d : ds) s += d.value; return s;
}
Device* find_by_id(vector<Device>& ds, const string& id) {
    for (auto& d : ds) if (d.id == id) return &d;
    return nullptr;
}
```

3. 固定容量读数窗口

考的是环形缓冲越界,下标要取模。

```
#include <stdio.h>
#define CAP 8
```

```
static int win[CAP];
static int head = 0, count = 0;

void add_reading(int v) {
    win[head] = v;
    head = (head + 1) % CAP;
    if (count < CAP) count++;
}
void print_readings(void) {
    int start = (count < CAP) ? 0 : head;
    for (int i = 0; i < count; i++)
        printf("%d\n", win[(start + i) % CAP]);
}
```

4. 多文件统计工具

考的是头文件保护、声明与实现一致、CMake 配好。

stats.h:

```
#pragma once
#include <vector>
double mean(const std::vector<double>& v);
```

stats.cpp:

```
#include "stats.h"
double mean(const std::vector<double>& v) {
    if (v.empty()) return 0.0;
    double s = 0; for (double x : v) s += x;
    return s / v.size();
}
```

CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.10)
project(stats CXX)
set(CMAKE_CXX_STANDARD 17)
add_executable(main main.cpp stats.cpp)
```

构建:

```
cmake -S . -B build && cmake --build build
```

总结与灵感

做完这一整套题,最深的体会可以归成几条。

第一,先看源码再动手。exam 看着是个黑盒,其实 cat 一下就是几行 Python,参数是 -e 还是 -k、密钥怎么算,一眼就看明白了。猜参数远不如读代码省时间。

第二,读懂校验条件就能反推输入。第 3 关补空格,表面看很绕,其实就是输入长度要能被密钥长度整除,(-N) % K 一个式子就出来了。很多看着复杂的规则,背后都是简单的数学。

第三,脚本要幂等。第 6、7 关都要求重复执行,诀窍就是每步操作前先判断在不在、有没有,存在就复用,不存在才创建,跑多少遍结果都一样。

第四,对待进程要温柔。判断存活用 kill -0 只探测不发信号,退出用 TERM 给进程收尾的机会,不误伤也不暴力。

第五,Git 要留痕。撤销用 revert 而不是 reset,历史链完整、可回溯,这是成熟的工程习惯。

第六,细节就是命。一个换行、一个文件名后缀、一个参数名、一个退出码,任何一个小地方错了都会全盘皆输。

关于灵感来源,我主要受了这么几样东西的影响。

一是 Unix 哲学,尤其是 KISS,能简单就别复杂。遇到问题先想有没有笨而稳的办法,这也是我很多关愿意用 printf 控制字节、用简单脚本而不是花活的原因。

二是 The Art of Unix Programming 里说的,把程序做成小工具、用管道和重定向把它们组合起来。第 2 关的 find 加 grep、第 4 关的 route.sh,都是这个思路。

三是 man page 和 --help 是最好的文档。第 1 关最后就是靠 --help 和 grep 源码把参数定下来的,吃透官方接口比瞎猜强太多。

四是数学里的取模。循环异或、环形缓冲、补空格,背后都是 mod 运算,这些初等数学在工程里是真的管用。

这套题不炫技,但每一步都算数,把基础打牢了,后面才走得远。